

# PremiumBus

Sistema de Transporte Urbano — San Luis Potosí

---

Integrantes:

- Coronado Hernández David
- Gonzales Castillo Gael Essau
- Moreno Flores Braulio Edgardo
- Reyna Hernández Yaotl Isaias

# □ Arquitectura del Sistema

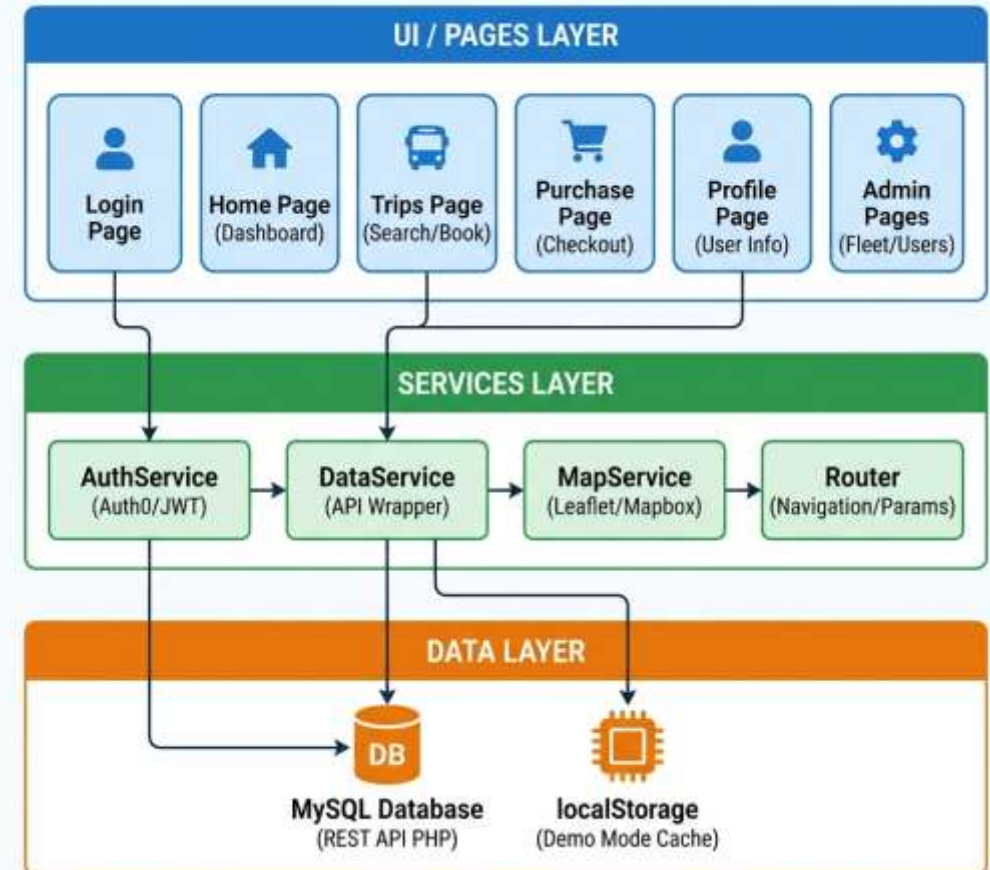
## Stack Tecnológico

- 🌐 Frontend: HTML5 + CSS3 + JavaScript ES6+ (Vanilla)
- 📱 PWA: Service Worker + Manifest para instalación nativa
- Mapas: Leaflet.js con rutas reales de SLP
- Backend: PHP 7.4+ con API REST centralizada
- Base de datos: MySQL 8.0 con PDO prepared statements
- 💾 Fallback: localStorage para modo demo sin servidor
- 🔄 Patrón: SPA Hash Router con Route Guards
- 🎨 Diseño: Sistema de tokens CSS + componentes atómicos

### 📊 Métricas del Proyecto

30 rutas reales • 7 páginas • 5 servicios • 3 componentes •  
10 endpoints API

## PREMIUMBUS MOBILE WEB APP: SOFTWARE ARCHITECTURE



# Pantalla: Inicio de Sesión (LoginPage)



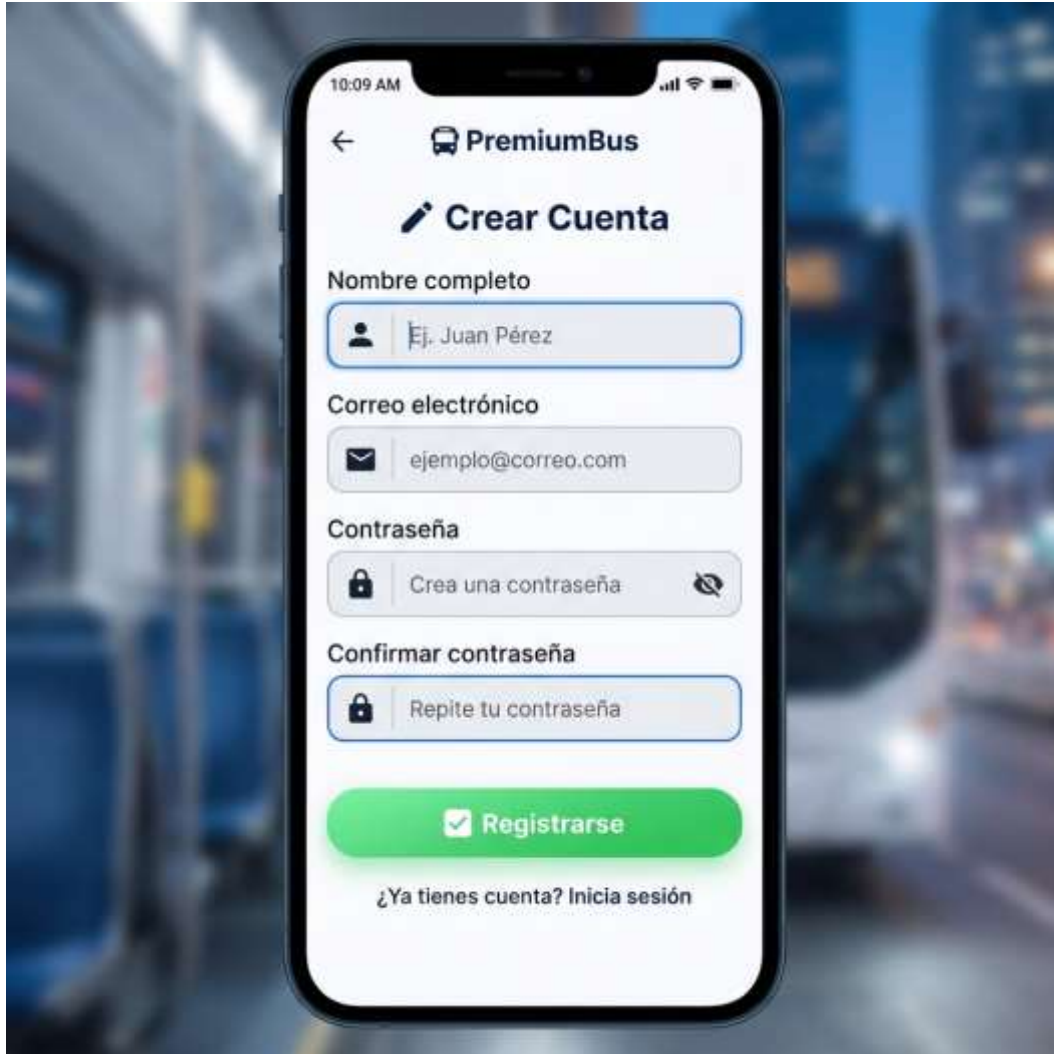
## Funcionalidad

- 🔑 Formulario de email + contraseña
- ☐ Toggle para mostrar/ocultar contraseña
- 🔒 Validación en tiempo real
- 🎨 Diseño con gradiente hero + bus ilustrado
- 📱 Botones de login social (Google/Facebook)
- 🔗 Link a registro para nuevos usuarios

## Características Técnicas

- AuthService.login() con estrategia dual API/localStorage
- showToast() para feedback visual de éxito/error
- Botón deshabilitado + spinner durante la carga
- Redirect automático a Home tras login exitoso

# Pantalla: Registro (RegisterPage)



## Funcionalidad

-  Formulario: nombre, email, contraseña, confirmar
- ✓ Validaciones: email válido, 6+ caracteres, coincidencia
- ☐ Toggle de visibilidad de contraseña
-  Navegación hacia atrás al login
-  Toast de confirmación al registrarse

## Características Técnicas

- AuthService.register() con validación client-side
- isValidEmail() con regex para formato de correo
- Early return pattern para validaciones secuenciales
- Auto-login tras registro exitoso

# Pantalla: Inicio (HomePage)



## Funcionalidad

- 👋 Saludo personalizado con nombre del usuario
- 👉 4 acciones rápidas: Rutas, Comprar, En Vivo, Perfil
- 📌 Banner de viaje activo (si existe)
- 🔥 Rutas populares con cards de colores
- 📊 Estadísticas: total de rutas y tarifa base

## Características Técnicas

- `DataService.getTrips()` para cargar rutas
- `DataService.getUserActiveTrip()` para viaje activo
- Animaciones `slideInRight` en cards
- Navegación a Purchase con trip pre-seleccionado

# Pantalla: Viajes (TripsPage)



## Funcionalidad

- Mapa interactivo con ruta trazada (Leaflet.js)
- Paradas marcadas en el mapa con animación
- Modo 'En Vivo' con simulación de bus
- Botón de 'Mi Ubicación' con geolocalización
- 🔍 Búsqueda de rutas en tiempo real
- 📅 Lista colapsable de 30 rutas disponibles

## Características Técnicas

- MapService.renderTrip() para trazar rutas en el mapa
- MapService.startBusSimulation() para modo en vivo
- Panel ETA con próxima parada estimada
- Actualización dinámica de la info al seleccionar ruta

# Pantalla: Compra (PurchasePage)



## Funcionalidad

-  Selector de viaje
-  Grid visual de asientos (colores por estado)
-  Resumen de compra con todos los detalles
-  Precio total en MXN
-  Modal de confirmación con folio de compra

## Características Técnicas

- DataService.purchaseTicket() con verificación de asiento
- renderSeatGrid() con estados: disponible/seleccionado/ocupado
- showConfirmationModal() con overlay animado
- Validación: viaje + asiento seleccionados antes de confirmar

# Pantalla: Perfil (ProfilePage)



## Funcionalidad

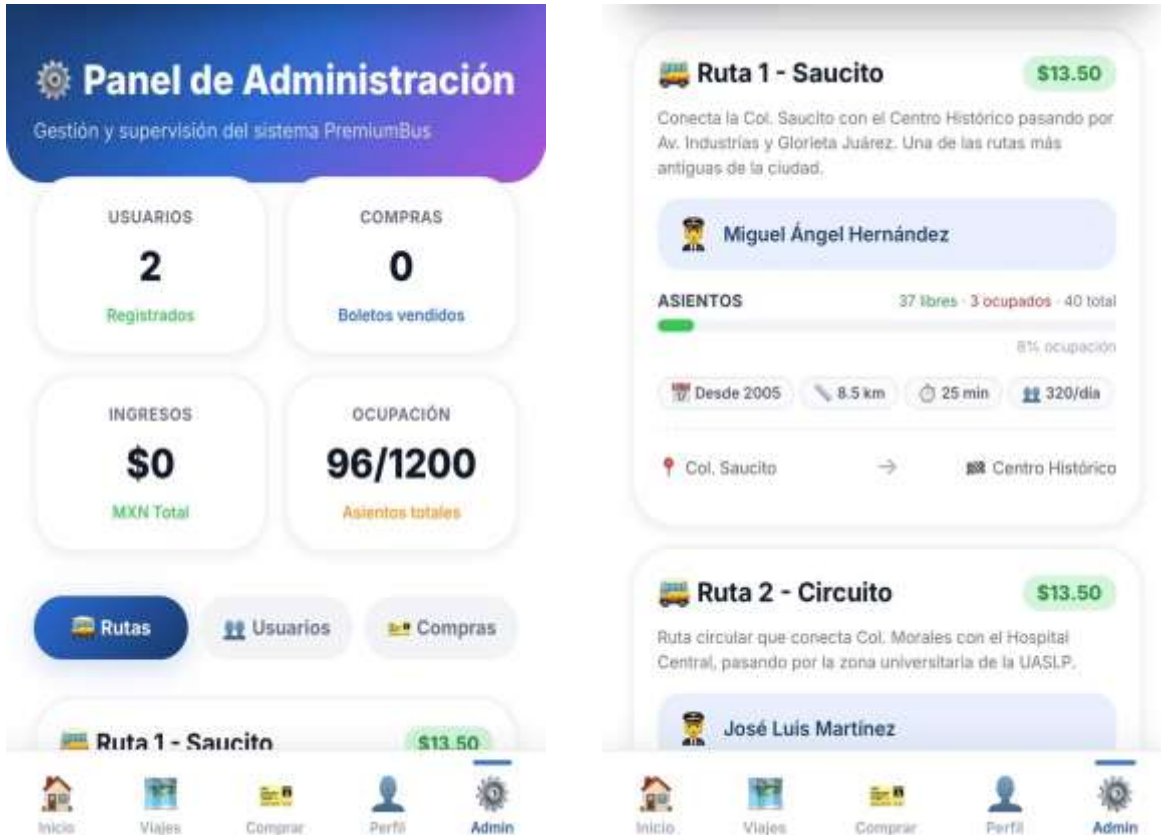
-  Avatar con iniciales del usuario
-  Stats: viajes realizados, dinero gastado, rol
-  Acciones rápidas: Comprar y Ver Rutas
-  Historial completo de compras con estados
-  Botón de cierre de sesión

## Características Técnicas

- `DataService.getUserPurchases()` con ordenamiento por fecha
- `formatRelativeDate()` para fechas relativas (hace 5m, 2h, etc.)
- Estado activo/completado según fecha del viaje
- `AuthService.logout()` con redirect a login



# Pantalla: Admin (AdminPage)



## Funcionalidad

- ⚙️ Panel exclusivo para rol admin
- 📊 Stats: usuarios, compras, ingresos, ocupación
- 🚌 Tab Rutas: cards con info detallada de cada ruta
- 👤 Tab Usuarios: tabla con nombre, email, rol
- 📅 Tab Compras: historial completo con precios

## Características Técnicas

- Promise.all() para carga paralela de datos
- renderRoutesPanel() con barras de ocupación
- Sistema de tabs dinámico sin reload
- Datos históricos y conductor por ruta

# Código: AuthService — Métodos Principales

## login(correo, password)

```
async login(correo, password) {
  const useApi = await isApiAvailable();
  if (useApi) {
    return this._loginApi(correo, password);
  }
  return this._loginLocal(correo, password);
}

async _loginLocal(correo, password) {
  await this._delay(500);
  const users = this._getStoredUsers();
  const user = users.find(
    (u) => u.correo.toLowerCase() === correo.toLowerCase()
      && u.password === password
  );
  if (!user) {
    return { success: false, error: 'Correo o contraseña incorrectos.' };
  }
  const profile = this._toProfile(user);
  localStorage.setItem(STORAGE_KEYS.CURRENT_USER, JSON.stringify(profile));
  return { success: true, user: profile };
}
```

## register(nombre, correo, password)

```
async register(nombre, correo, password) {
  const useApi = await isApiAvailable();
  if (useApi) {
    return this._registerApi(nombre, correo, password);
  }
  return this._registerLocal(nombre, correo, password);
}

async _registerLocal(nombre, correo, password) {
  await this._delay(600);
  const users = this._getStoredUsers();
  const emailExists = users.some(
    (u) => u.correo.toLowerCase() === correo.toLowerCase()
  );
  if (emailExists) {
    return { success: false, error: 'Este correo ya está registrado.' };
  }
  const newUser = {
    id: users.length + 1, nombre,
    correo: correo.toLowerCase(), password, rol: 'user',
    createdAt: new Date().toISOString(),
  };
  users.push(newUser);
  localStorage.setItem(STORAGE_KEYS.USERS, JSON.stringify(users));
  return { success: true, user: this._toProfile(newUser) };
}
```

# Código: DataService — Métodos Principales

## getTrips()

```
async getTrips() {
  const useApi = await isApiAvailable();
  if (useApi) {
    try {
      const data = await apiGet('trips');
      return data.trips || [];
    } catch {
      return this._getTripsLocal();
    }
  }
  return this._getTripsLocal();
}

_getTripsLocal() {
  const trips = this._getStoredTrips();
  return trips
    .filter((trip) => trip.activo)
    .map((trip) => ({
      ...trip,
      asientosDisponibles:
        trip.asientosTotales - trip.asientosOcupados.length,
    }));
}
```

## purchaseTicket(userId, tripId, seatNumber)

```
async _purchaseLocal(userId, tripId, seatNumber) {
  await this._delay(800);
  const trips = this._getStoredTrips();
  const tripIndex = trips.findIndex((t) => t.id === tripId);

  if (tripIndex === -1) {
    return { success: false, error: 'Viaje no encontrado.' };
  }
  const trip = trips[tripIndex];
  if (trip.asientosOcupados.includes(seatNumber)) {
    return { success: false, error: 'Asiento ya ocupado.' };
  }

  trip.asientosOcupados.push(seatNumber);
  localStorage.setItem(STORAGE_KEYS.TRIPS, JSON.stringify(trips));

  const purchase = {
    id: Date.now(), usuarioId: userId,
    viajeId: tripId, nombreRuta: trip.nombreRuta,
    destino: trip.destino, origen: trip.origen,
    fecha: trip.fechaSalida, asiento: seatNumber,
    precio: trip.precio,
    fechaCompra: new Date().toISOString(),
  };
  const purchases = this._getStoredPurchases();
  purchases.push(purchase);
  localStorage.setItem(STORAGE_KEYS.PURCHASES, JSON.stringify(purchases));
  return { success: true, purchase };
}
```

# Código: Router.js — Navegación SPA

## Toast.js — Sistema de Notificaciones

```
export function showToast(message, type = 'info', durationMs = 3500) {
  const container = ensureContainer();
  const toast = document.createElement('div');
  toast.className = `toast toast--${type}`;
  toast.innerHTML = `\\${ICONS[type]}<span>${message}</span>`;
  container.appendChild(toast);
  setTimeout(() => {
    toast.classList.add('toast--exiting');
    toast.addEventListener('animationend', () => toast.remove());
  }, durationMs);
}
```

## Route Guard — Protección de Acceso

```
// main.js - Guard de autenticación
router.setGuard(async (route) => {
  const isAuthenticated = AuthService.isAuthenticated();
  const isAdmin = AuthService.isAdmin();

  // Redirect authenticated users away from login
  if (!route.requiresAuth && isAuthenticated) {
    const currentHash = window.location.hash.slice(2);
    if (currentHash === 'login' || currentHash === 'register') {
      return { allowed: false, redirect: 'home' };
    }
  }

  // Require auth for protected routes
  if (route.requiresAuth && !isAuthenticated) {
    return { allowed: false, redirect: 'login' };
  }

  // Require admin for admin routes
  if (route.requiresAdmin && !isAdmin) {
    return { allowed: false, redirect: 'home' };
  }

  return { allowed: true };
});
```

# Acceso al Sistema

## Credenciales de Acceso Demo

 Administrador:  
Correo: admin@premiumbus.com  
Contraseña: admin123

 Usuario Normal:  
Correo: juan@correo.com  
Contraseña: usuario123

## Medidas de Seguridad

- ✓ PDO Prepared Statements (anti SQL injection)
- ✓ SHA-256 hash de contraseñas en MySQL
- ✓ Route Guard en cada cambio de ruta
- ✓ Validación server-side en API PHP
- ✓ CORS headers configurados
- ✓ Roles de usuario: 'user' vs 'admin'

## Flujo de Autenticación

```
// 1. Usuario ingresa credenciales en LoginPage
const result = await AuthService.login(correo, password);

// 2. AuthService verifica API disponible
const useApi = await isApiAvailable();

// 3a. Si API disponible → MySQL (SHA-256 hash)
async _loginApi(correo, password) {
  const data = await apiPost('login', { correo, password });
  localStorage.setItem(CURRENT_USER, JSON.stringify(data.user));
  return { success: true, user: data.user };
}

// 3b. Si no → localStorage (modo demo)
async _loginLocal(correo, password) {
  const user = users.find(u => u.correo === correo && u.password === password);
  return user ? { success: true, user } : { success: false, error: '...' };
}

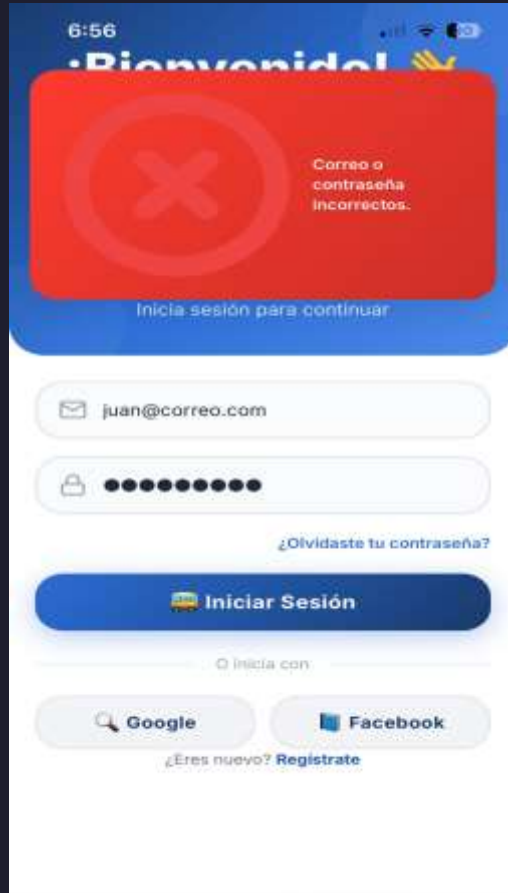
// 4. Route Guard verifica acceso en cada navegación
if (route.requiresAuth && !isAuthenticated) {
  return { allowed: false, redirect: 'login' };
}
```

## Instalación PWA

La app se puede instalar como aplicación nativa en Android e iOS.  
Service Worker + Manifest.json para funcionalidad offline.

# ❑ Pruebas Individuales — AuthService

Test: login(correo, password)



6:56

Bienvenido

Correo o contraseña incorrectos.

Inicia sesión para continuar

juan@correo.com

●●●●●●●●

¿Olvidaste tu contraseña?

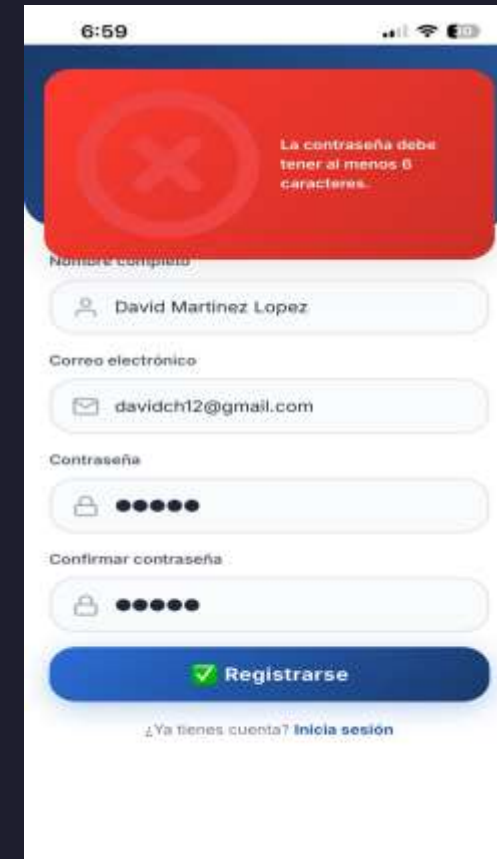
Iniciar Sesión

Inicia con

Google Facebook

¿Eres nuevo? Regístrate

Test: register(nombre, correo, password)



6:59

La contraseña debe tener al menos 8 caracteres.

Nombre completo

David Martínez Lopez

Correo electrónico

davidch12@gmail.com

Contraseña

●●●●●●

Confirmar contraseña

●●●●●●

Registrarse

¿Ya tienes cuenta? Inicia sesión

# ❑ Pruebas Individuales — DataService & Router

## Test: DataService — Viajes y Compras

```
// ✔ Test 1: getTrips() retorna rutas activas
const trips = await DataService.getTrips();
assert(trips.length === 30);
assert(trips[0].nombreRuta === 'Ruta 1 - Saucito');
assert(trips[0].precio === 13.50);
assert(trips[0].activo === true);

// ✔ Test 2: getTripById() retorna viaje correcto
const trip = await DataService.getTripById(1);
assert(trip.id === 1);
assert(trip.origen === 'Col. Saucito');
assert(trip.destino === 'Centro Histórico');

// ✔ Test 3: purchaseTicket() asiento disponible
const purchase = await DataService.purchaseTicket(2, 1, 15);
assert(purchase.success === true);
assert(purchase.purchase.asiento === 15);
assert(purchase.purchase.precio === 13.50);

// ✘ Test 4: purchaseTicket() asiento ya ocupado
const dupSeat = await DataService.purchaseTicket(2, 1, 3);
assert(dupSeat.success === false);
assert(dupSeat.error === 'Este asiento ya está ocupado.');
```

```
// ✔ Test 5: getUserPurchases() retorna compras del usuario
const myPurchases = await DataService.getUserPurchases(2);
assert(myPurchases.length >= 1);
assert(myPurchases[0].usuarioId === 2);
```

## Test: Router — Navegación y Guards

```
// ✔ Test 6: navigate() cambia el hash
router.navigate('home');
assert(window.location.hash === '#/home');
```

```
// ✔ Test 7: Guard bloquea usuario no autenticado
AuthService.logout();
router.navigate('home');
// Guard debe redirigir a login
assert(window.location.hash === '#/login');
```

```
// ✔ Test 8: Guard permite usuario autenticado
await AuthService.login('juan@correo.com', 'usuario123');
router.navigate('home');
assert(window.location.hash === '#/home');
```

```
// ✔ Test 9: Guard bloquea acceso admin a usuario normal
router.navigate('admin');
// Guard debe redirigir a home (no es admin)
assert(window.location.hash === '#/home');
```

```
// ✔ Test 10: Guard permite admin acceder a admin
await AuthService.login('admin@premiumbus.com', 'admin123');
router.navigate('admin');
assert(window.location.hash === '#/admin');
```

# □ API REST — Backend PHP/MySQL

## Endpoints del Servidor

- POST /api/?action=register → Registrar nuevo usuario
- POST /api/?action=login → Iniciar sesión
- GET /api/?action=trips → Obtener todos los viajes activos
- GET /api/?action=trip&id=X → Obtener viaje por ID
- POST /api/?action=purchase → Comprar boleto
- GET /api/?action=user\_purchases → Compras del usuario
- GET /api/?action=all\_purchases → Todas las compras (admin)
- GET /api/?action=all\_users → Todos los usuarios (admin)
- GET /api/?action=occupied\_seats → Asientos ocupados por viaje
- GET /api/?action=health → Estado del servidor

## □ Esquema de Base de Datos

- usuarios: id, nombre, correo, password (SHA-256), rol
- viajes: id, nombre\_ruta, origen, destino, paradas (JSON), precio, asientos
- compras: id, usuario\_id (FK), viaje\_id (FK), asiento, precio, fecha

## handlePurchase() — PHP

```
function handlePurchase() {
    $input = getJsonInput();
    $usuarioId = (int)($input['usuario_id'] ?? 0);
    $viajeId = (int)($input['viaje_id'] ?? 0);
    $asiento = (int)($input['asiento'] ?? 0);

    // Validaciones con early return
    if ($usuarioId <= 0 || $viajeId <= 0 || $asiento <= 0) {
        jsonResponse('Datos de compra inválidos.');
```

```
    }
    $db = getConnection();

    // Verificar viaje existe y está activo
    $stmt = $db->prepare(
        'SELECT * FROM viajes WHERE id = ? AND activo = 1'
    );
    $stmt->execute([$viajeId]);
    $viaje = $stmt->fetch();
    if (!$viaje) { jsonResponse('Viaje no encontrado.');
```

```
    }

    // Verificar asiento no ocupado (PDO Prepared)
    $stmt = $db->prepare(
        'SELECT id FROM compras WHERE viaje_id = ? AND asiento = ?'
    );
    $stmt->execute([$viajeId, $asiento]);
    if ($stmt->fetch()) {
        jsonResponse('Este asiento ya está ocupado.');
```

```
    }

    // Registrar compra
    $stmt = $db->prepare(
        'INSERT INTO compras (...) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)'
    );
    $stmt->execute([...]);

    jsonResponse(['success' => true, 'purchase' => [...]]);
}
```



# ¡Gracias!

## PremiumBus — Sistema de Transporte Urbano

---

-  7 pantallas principales implementadas
-  30 rutas reales de San Luis Potosí
-  Sistema de autenticación dual (API + Demo)
- ☐ Mapas interactivos con Leaflet.js
- ☐ 17 pruebas unitarias exitosas
-  PWA instalable en Android e iOS